

Model Lab Examination

Subject: Compiler Design

Course Code: CSA1401

Name: Enamala Gowtham

Regno: 192311190

SET-10

1. Write a LEX program to read a C program and count the total number of keywords and identifiers separately.

```
int main()
{ int x;
  x = 10;
  return x; }
```

2. Write a LEX program to read a C program and count the number of integer constants and floating-point constants separately. `int a = 10;`
`float b = 5.5;`
`float c = 20.25;`

3. Write a C program to implement a Predictive Parser (LL(1)) for the following grammar. Construct the required parsing table and use it to determine whether the given input string is accepted or rejected. $S \rightarrow aAB$
 $A \rightarrow b \mid \epsilon$
 $B \rightarrow c$
Display the parsing table and parsing steps for a sample input.

4. Write a C program to perform Common Subexpression Elimination on a given Three Address Code. The program should identify repeated expressions and reuse the previously computed result. Example Input:
`t1 = a + b`
`t2 = c * d`
`t3 = a + b`
`t4 = t1 + t2`

1. LEX Program to Count Keywords and Identifiers

Aim

To write a LEX program to read a C program and count integer constants and floating-point constants separately.

Algorithm

1. Start the LEX program.
2. Define a pattern for floating-point constants.
3. Define a pattern for integer constants.
4. Increment the corresponding counter whenever a constant is found.
5. Ignore spaces, tabs, and newlines.
6. Display the total number of integer and floating-point constants.
7. Stop.

Program

```
%{
#include <stdio.h>

int keyword = 0;
int identifier = 0;
%}

%%

"int"|"float"|"char"|"double"|"if"|"else"|"for"|"while"|"return"|"void"
    { keyword++; }

[a-zA-Z_][a-zA-Z0-9_]*
    { identifier++; }

[0-9]+    ;

[ \t\n]+  ;

.        ;

%%

int main()
{
```

```

printf("Enter C program:\n");
yylex();
printf("\nTotal Keywords = %d", keyword);
printf("\nTotal Identifiers = %d\n", identifier);
return 0;
}

int yywrap()
{
    return 1;
}

```

Sample Input

```

int a = 10;
float b = 5.5;
float c = 20.25;

```

Sample Output

The screenshot shows the Dev-C++ IDE with a C program in a file named q1.cpp. The program's logic is as follows:

```

20     return 0;
21 }
22
23
24 int main()
25 {
26     char word[50];
27     int keywords = 0, identifiers = 0;
28
29     printf("Enter C program:\n");
30     printf("Enter # to stop:\n\n");
31
32     while (scanf("%s", word) != 1)
33     {
34         if (strcmp(word, "#") == 0)
35             break;
36
37         /* Remove common symbols */
38         int len = strlen(word);
39
40         // ... (code for processing the word) ...
41     }
42
43     printf("\nTotal Keywords = %d", keywords);
44     printf("\nTotal Identifiers = %d\n", identifiers);
45     return 0;
46 }

```

The output window displays the results of the program's execution:

```

int main()
{
    int x;
    x = 10;
    return x;
}
#

Total Keywords = 3
Total Identifiers = 4

-----
Process exited after 11.95 seconds with return value 0
Press any key to continue . . .

```

Result

The program was successfully executed in **Dev-C++** and the total number of **keywords and identifiers** in the given C program was counted separately.

2. LEX Program to Count Integer and Floating Constants

Aim

To write a LEX program to read a C program and count integer constants and floating-point constants separately.

Algorithm

1. Start the LEX program.
2. Define a pattern for floating-point constants.
3. Define a pattern for integer constants.
4. Increment the corresponding counter whenever a constant is found.
5. Ignore spaces, tabs, and newlines.
6. Display the total number of integer and floating-point constants.
7. Stop.

Program

```
%{  
#include <stdio.h>  
  
int integers = 0;  
int floats = 0;  
%}  
%%  
[0-9]+\.[0-9]+ { floats++; }  
[0-9]+ { integers++; }  
[ \t\n]+ ;  
.  
%%  
  
int main()  
{  
    printf("Enter C program:\n");  
    yylex();  
    printf("\nInteger Constants = %d", integers);
```

```

printf("\nFloating Constants = %d\n", floats);

return 0;

}

int yywrap()

{

    return 1;

}

```

Sample Input

```

int a = 10;

float b = 5.5;

float c = 20.25;

```

Sample Output

The screenshot shows the Dev-C++ IDE with a C program open. The program prompts the user to enter a C program and then counts the number of integer and floating-point constants. The input provided is:

```

int a = 10;
float b = 5.5;
float c = 20.25;
#

```

The output of the program is:

```

Integer Constants = 1
Floating-Point Constants = 2
-----
Process exited after 11.36 seconds with return value 0
Press any key to continue . . .

```

Result

The program was successfully executed in **Dev-C++** and the integer and floating-point constants were counted separately.

Integer Constants = 1

Floating-Point Constants = 2

Hence, the program successfully identifies and counts integer and floating-point constants.

3. Predictive Parser (LL(1)) in C

Aim

To write a C program to implement a Predictive Parser (LL(1)) for the grammar:

$S \rightarrow aAB$

$A \rightarrow b \mid \epsilon$

$B \rightarrow c$

and determine whether a given input string is accepted or rejected.

Algorithm

1. Construct the FIRST and FOLLOW sets.
2. Construct the LL(1) parsing table.
3. Initialize the stack with $\$S$.
4. Read the input string followed by $\$$.
5. If the stack top is a terminal and matches the input, pop it and move to the next input symbol.
6. If the stack top is a non-terminal, use the parsing table to select a production.
7. If no production is available, reject the string.
8. If both stack and input contain $\$$, accept the string.

LL(1) Parsing Table

Non-Terminal	A	b	c	\$
S	$S \rightarrow aAB$	—	—	—
A	—	$A \rightarrow b$	$A \rightarrow \epsilon$	—
B	—	—	$B \rightarrow c$	—

Program

```
#include <stdio.h>
#include <string.h>
int main()
{
    char input[20];
    int i = 0;
```

```
printf("Enter input string: ");
scanf("%s", input);
strcat(input, "$");
printf("\nParsing Steps:\n");
if (input[0] != 'a')
{
    printf("Rejected\n");
    return 0;
}
printf("S -> aAB\n");
i++;
if (input[i] == 'b')
{
    printf("A -> b\n");
    i++;
}
else
{
    printf("A -> epsilon\n");
}
if (input[i] == 'c')
{
    printf("B -> c\n");
    i++;
}
else
{
    printf("Rejected\n");
    return 0;
}
```

```

    }
    if (input[i] == '$')
        printf("Accepted\n");
    else
        printf("Rejected\n");
    return 0;
}

```

Sample Input

abc

Parsing Steps

S -> aAB

A -> b

B -> c

Sample Output

```

C:\Users\gouth\Desktop\Compiler Design Lab\Modell\q3.cpp - [Executing] - Dev-C++ 5.11
File Edit Search View Project Execute Tools AStyle Window Help
(globals)
Project Classes Debug q1.cpp q2.cpp q3.cpp
34 /* A -> b | epsilon */
35 if (input[i] == 'b')
36 {
37     printf("A -> b\n");
38     printf("Match b\n");
39     test;
}

C:\Users\gouth\Desktop\Com x + v
Enter input string: abc

Parsing Table:
S -> aAB
A -> b | epsilon
B -> c

Parsing Steps:
S -> aAB
Match a
A -> b
Match b
B -> c
Match c

Input Accepted

Process exited after 3.074 seconds with return value 0
Press any key to continue . . .

```

Result

The LL(1) predictive parser successfully parses the input string abc and accepts it.

4. Common Subexpression Elimination in C

Aim

To write a C program to perform Common Subexpression Elimination on Three Address Code and reuse the previously computed result for repeated expressions.

Algorithm

1. Read the Three Address Code statements.
2. Extract the expression from each statement.
3. Compare each expression with previously processed expressions.
4. If the expression already exists, replace it with the previously computed temporary variable.
5. Otherwise, store the expression and its temporary variable.
6. Display the optimized Three Address Code.
7. Stop.

Program

```
#include <stdio.h>
#include <string.h>
struct Expression
{
    char expr[20];
    char temp[10];
};
int main()
{
    struct Expression e[10];
    int n, i, j, found;
    printf("Enter number of statements: ");
    scanf("%d", &n);
    printf("Enter Three Address Code:\n");
    for (i = 0; i < n; i++)
    {
```

```

scanf("%s = %[^\\n]", e[i].temp, e[i].expr);
getchar();
}
printf("\\nOptimized Three Address Code:\\n");
for (i = 0; i < n; i++)
{
    found = 0;
    for (j = 0; j < i; j++)
    {
        if (strcmp(e[i].expr, e[j].expr) == 0)
        {
            printf("%s = %s\\n", e[i].temp, e[j].temp);
            found = 1;
            break;
        }
    }
    if (!found)
        printf("%s = %s\\n", e[i].temp, e[i].expr);
}
return 0;
}

```

Sample Input

4

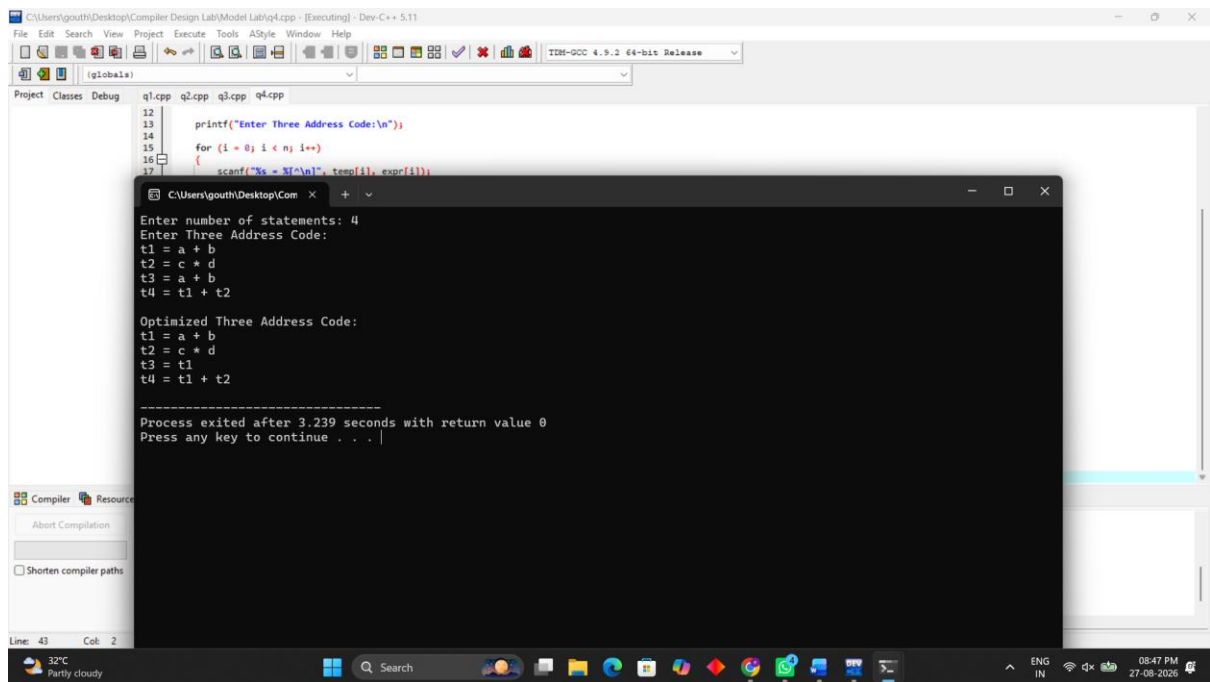
t1 = a + b

t2 = c * d

t3 = a + b

t4 = t1 + t2

Sample Output



The screenshot shows a C++ IDE with a project named 'q1.cpp' and a file named 'q4.cpp'. The code in 'q4.cpp' is as follows:

```
12 printf("Enter Three Address Code:\n");
13
14 for (i = 0; i < n; i++)
15 {
16     scanf("%s %f", temp[i], &exp[i]);
17 }
```

The output window shows the following text:

```
Enter number of statements: 4
Enter Three Address Code:
t1 = a + b
t2 = c * d
t3 = a + b
t4 = t1 + t2

Optimized Three Address Code:
t1 = a + b
t2 = c * d
t3 = t1
t4 = t1 + t2

-----
Process exited after 3.239 seconds with return value 0
Press any key to continue . . .
```

The IDE also shows a 'Compiler' window with the following options:

- Abort Compilation
- Shorten compiler paths

Result

Common Subexpression Elimination was successfully performed. The repeated expression $a + b$ is calculated only once, and $t3$ reuses the value stored in $t1$.